ASSIGNMENT 1

Course: B.Tech. CSE **Semester:5**

Subject: System Design **Subject Code:**

TCS-504 Submission Date: 07-Septemeber-2026

1. Problem

Build a small movie ticket booking system for a single cinema (like PVR or INOX), as a menu-driven C++ console program.

A customer should be able to see which movies are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

2. Scope

Features	Description
F1	List all movies currently playing.
F2	For a chosen movie, list its shows (screen + start time).
F3	For a chosen show, display the seat layout with available / booked status.
F4	Book one or more seats for a show (reject an already-booked seat).
F5	Price the booking by seat type: silver ₹150, gold ₹250, platinum ₹400.
F6	Pay by UPI, card or cash — a failed payment must not confirm the booking.
F7	Print a ticket: booking id, movie, screen, time, seat numbers, total amount.
F7	Cancel a booking — the seats become available again.

3. Requirement analysis

a) Functional Requirements (FR)

Functional Requirements	Description
FR 1	**Movie listing:** The system displays all movies currently marked as "currently playing," showing at minimum title, language, and duration for each. If no movies are playing, the system displays an appropriate empty-state message instead of an empty or broken list.
FR 2	**Show listing:** Given a selected movie, the system lists all shows for that movie, where each show entry displays the screen number and start time. If the movie has zero shows scheduled, the system informs the customer rather than showing a blank menu.
FR 3	**Seat layout display:** Given a selected show, the system displays every seat in that show's screen along with its current status (AVAILABLE or BOOKED), grouped by seat type (SILVER/GOLD/PLATINUM). The layout reflects only that show's ShowSeat records — booking status is never shared across shows on the same screen.
FR 4	**Seat Selection & Lock:** A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.
FR 5	**Pricing:** The system calculates the total booking amount by summing each selected seat's price according to its type: SILVER = ₹150, GOLD = ₹250, PLATINUM = ₹400. The calculation must use named constants, not hardcoded literals, and must be independent of how the seats were selected.
FR 6	**Payment Handling:** Exactly one method (UPI/Card/Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.
FR 7	**Ticket printing:** After a booking is CONFIRMED (payment succeeded), the system prints a ticket containing the booking ID, movie title, screen number, show start time, the list of booked seat numbers, and the total amount paid. A ticket must never be printed for a booking whose status is not CONFIRMED.
FR 8	**Cancellation:** Given a valid, existing booking ID, the system cancels that booking, sets its status to CANCELLED, and changes every ShowSeat associated with it back to AVAILABLE. Attempting to cancel a booking ID that doesn't exist, or one already cancelled, must produce a clear error message and change nothing.

b) Non-Functional Requirements (NFR)

Non-Functional Requirements	Description
NFR 1	**(Modularity)** — Each class is implemented in its own file with no header files, and each class exposes only the methods other classes need (no public fields for internal state).
NFR 2	**(Extensibility)** — Adding a new payment method (e.g., NetBanking) must be achievable by adding one new class that extends `Payment`, without modifying `BookingService`, `Booking`, or any existing class.
NFR 3	**(Robustness/Input validation)** — Any invalid input — an out-of-range menu choice, a malformed seat code, a non-existent show number — must be caught and reported with a clear message, and must never crash the program or corrupt existing booking/seat state.
NFR 4	**Transactional State Consistency:** If a booking fails or a payment is declined midway, all temporarily selected seats must immediately revert to AVAILABLE [ ] so no seat stays locked incorrectly.
NFR 5	**Unique ID Generation:** Use a shared static counter inside the Booking class to auto-increment unique IDs (e.g., BK1001, BK1002) so no two tickets ever share the same identifier.

4. Noun-Verb Analysis

a) Noun Analysis (Classes & Attributes)

Noun Found	Keep as Class?	Reason
Movie	Yes	Has its own identity, title, language, and duration.
Seat	Yes	Represents a physical chair with a seat number and category (SILVER, GOLD, PLATINUM).
"seat layout"	No	It is a printed visual view of a Show's seat grid, handled by a method inside Show or BookingService.
Cinema	Yes	Top-level entity that owns and manages auditorium screens.
Screen	Yes	Represents an auditorium hall that owns a physical set of seats.

Show	Yes	Binds a Movie to a Screen at a specific time slot.

ShowSeat	Yes	Tracks the dynamic state (AVAILABLE or BOOKED) of a physical seat for a specific show.
Customer	Yes	Holds user details such as name and contact phone number.
Booking	Yes	Stores a confirmed or failed transaction record (Booking ID, show, seats, amount, status).
Payment	Yes	Abstract base entity that defines the payment interface (pay(amount)).
UPI / Card / Cash	Yes	Derived concrete classes implementing specific payment methods.
Price Calculator	Yes	Service class dedicated solely to calculating ticket prices based on seat tiers.
Ticket Printer	Yes	Service class dedicated strictly to formatting and displaying ticket receipts.
Booking Service	Yes	Core orchestrator managing the business workflow (booking, payment validation, cancellation).
"booking id"	No	Primitive attribute (string) inside the Booking class, not an independent entity.
"amount/ price"	No	Primitive numerical property (double), managed by PriceCalculator and Booking.
"menu/ choice"	No	Console UI interaction mechanism, handled inside main.cpp.

b) Verb Analysis (Methods & Operations)

Verb Found	Associated Class	Method Name	Responsibility
"list movies"	BookingService	getMovies() / displayMovies()	Fetches active movies from cinema inventory.
"list shows"	BookingService	getShowsForMovie()	Filters and returns shows matching a movie.
"display layout"	Show	displaySeatLayout()	Prints the matrix of seats showing AVAILABLE vs BOOKED.
"book seats"	ShowSeat / BookingService	lockSeat() / createBooking()	Marks selected seats as BOOKED if currently free.

"calculate price"	PriceCalculator	calculateTotal()	Sums prices of chosen seat types (SILVER, GOLD, PLATINUM).

"pay"	Payment	pay(double amount)	Executes transaction logic for UPI, Card, or Cash.
"print ticket"	TicketPrinter	printTicket()	Formats and outputs confirmed booking receipt.
"cancel booking"	BookingService / Booking	cancelBooking() / cancel()	Changes booking status to CANCELLED and frees seats back to AVAILABLE.

5. Classes Required

These are the classes required.

a. Core Entity Classes

Class	It's One Responsibility
Movie	Title, language, and duration — nothing else.
Seat	One physical seat: seat number and category type (SILVER/GOLD/PLATINUM) — nothing else.
Screen	Screen identifier and its collection of physical seats — nothing else.
Cinema	Theater name and its list of screens — nothing else.
Show	One specific time slot mapping a Movie to a Screen with its seat availability layout — nothing else.
ShowSeat	The runtime availability status (AVAILABLE vs BOOKED) of one physical seat for one specific show — nothing else.
Customer	User identification details (customer name and phone number) — nothing else.
Booking	Transaction record holding Booking ID, Customer, Show, booked seats, total amount, and booking status — nothing else.

b. Behavior and Service Classes

Class	It's One Responsibility

Payment	Abstract base contract declaring the pure virtual pay() method for payment methods — nothing else.
PaymentTypes (UpiPayment / CardPayment / CashPayment)	Concrete execution logic for processing payments via UPI, Card, or Cash — nothing else.

PriceCalculator	Pure calculation logic that computes the total booking cost based on seat categories — nothing else.
TicketPrinter	Formatting and displaying the ticket receipt on the console — nothing else.
BookingService	Orchestrating the core business workflow for seat reservation, payment execution, and cancellation — nothing else.

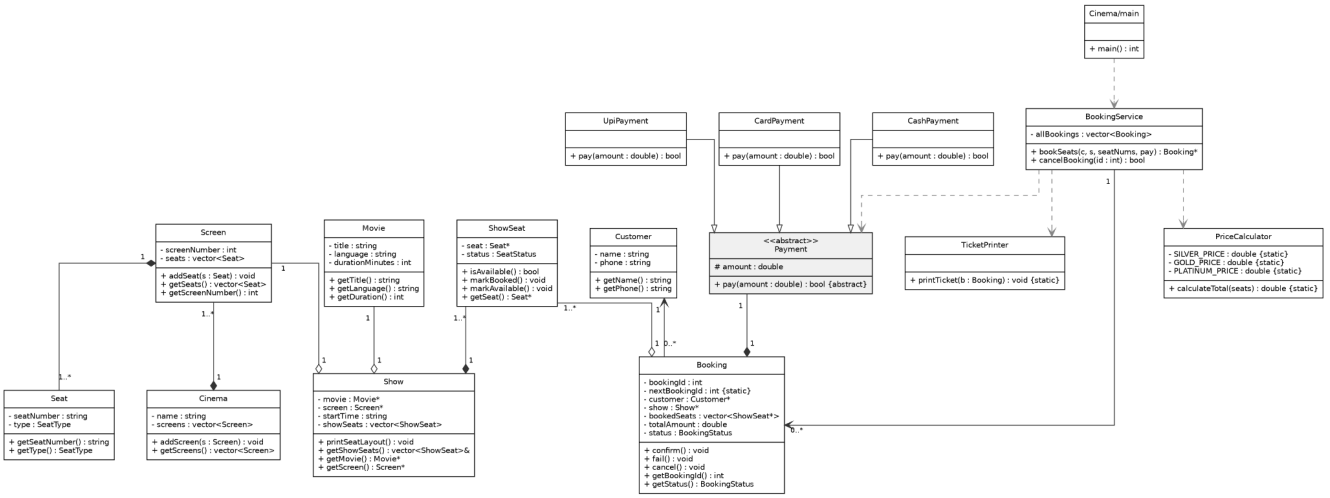
7. Relationships

PAIR	RELATION	JUSTIFICATION
Cinema → Screen	Composition	A Screen has no meaning outside its Cinema. If the Cinema is torn down, its Screens don't exist independently anywhere else. Cinema creates and owns its Screens in its constructor/setup; destroying the Cinema destroys the Screens with it.
Screen → Seat	Composition	A Seat is a fixed physical chair bolted inside one specific Screen. It has no identity or existence outside that Screen — if the Screen is demolished, seat A1 in it ceases to exist. Screen creates its Seats internally.
Show → Movie	Aggregation	A Show borrows a Movie. Cancel the 6 PM show and "3 Idiots" still exists and still plays at 9 PM. Show stores a reference and never deletes it.
Show → Screen	Aggregation	A Show *uses* a Screen at a given time slot, but the Screen exists before and after that Show and is reused by other Shows. Deleting the 6 PM Show doesn't delete Screen-1 — it's still there for the 9 PM Show.

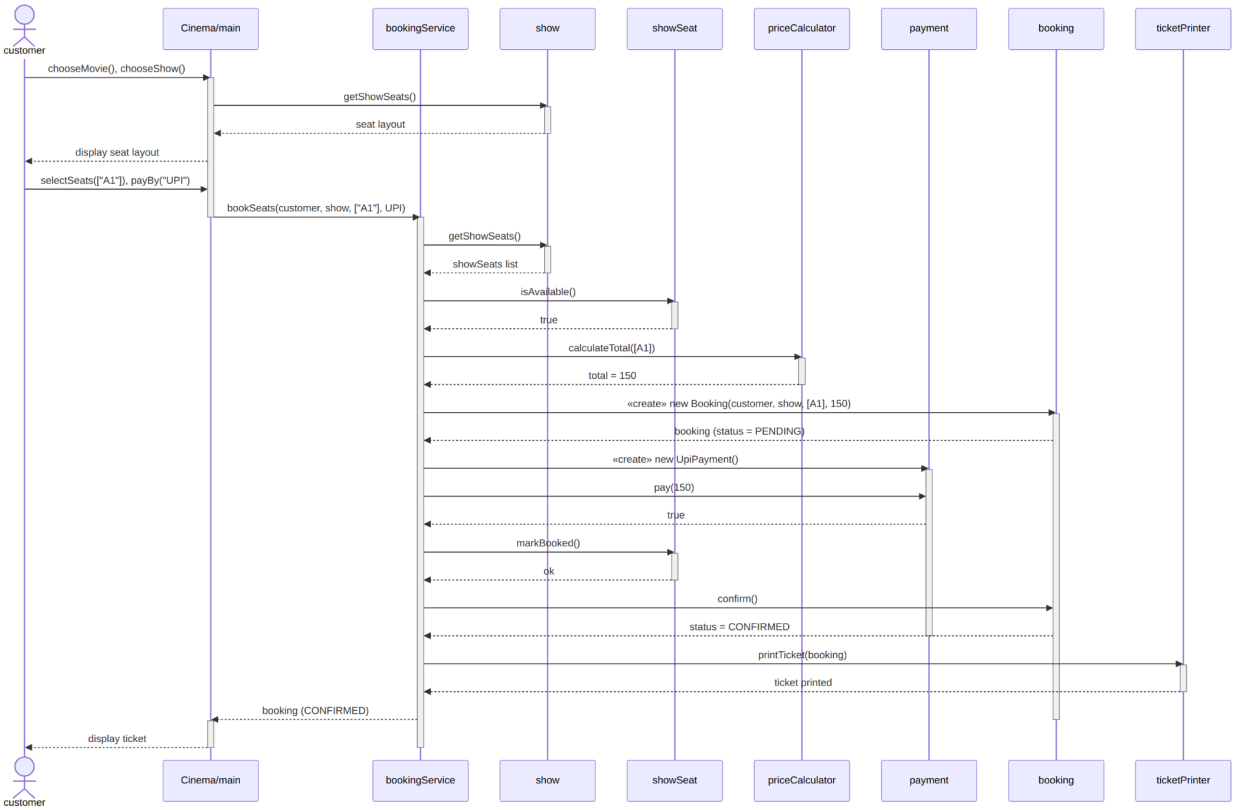
Show → ShowSeat	Composition	A ShowSeat's entire reason for existing is to track one seat's status *for this one Show*. It cannot exist for any other Show and has no meaning outside it. If the Show is deleted, all its ShowSeats are deleted with it — there's nothing left to track.
Booking → Customer	Aggregation	A Customer books a Booking, but neither owns the other's lifecycle. The Customer exists before and after any given Booking (they can make other bookings, or none at all), and deleting a Booking doesn't delete the Customer. It's a "uses/interacts with" link, not ownership either way.
Booking → ShowSeat	Aggregation	A Booking references the specific ShowSeats it booked, but those ShowSeats belong to (and are owned by) the Show, not the Booking. If a Booking is cancelled/deleted, the ShowSeats don't get destroyed — they just flip back to AVAILABLE and continue existing as part of the Show.
Booking → Payment	Association	A Payment record here is created specifically *for* one Booking's transaction and has no independent existence or reuse elsewhere. If the Booking is discarded, its Payment attempt is discarded with it — you don't reattach an old Payment object to a new Booking.

Payment → UpiPayment, CardPayment, CashPayment)	Inheritance	UpiPayment *is-a* concrete implementation of the abstract Payment class (UpiPayment , CardPayment, CashPayment extends Payment).
BookingService → Booking	Aggregation	BookingService manages a list of Booking records. The bookings exist as business domain records independently of the service

8.Class diagram



6. Sequence diagram



10. Modular code

```
CinemaBookingSystem/
├── Seat.cpp
├── Screen.cpp
├── Movie.cpp
├── ShowSeat.cpp
├── Show.cpp
├── Customer.cpp
├── Payment.cpp
├── UpiPayment.cpp
├── CardPayment.cpp
├── CashPayment.cpp
├── Booking.cpp
├── BookingService.cpp
├── TicketPrinter.cpp
└── main.cpp
```

A. Verification Checklist against Requirements

Requirement	Implementation Details	Status
One class per file	Every class (Seat, Screen, Movie, etc.) lives in its own standalone .cpp file.	In progress.
No header files	.cpp files include implementation natively; #include "*.cpp" in main.cpp allows compilation without .h headers.	In progress.
Encapsulation	seatStatus, bookingAmount, status are private and altered via dedicated guard methods (lockSeat, confirmSeat, setStatus).	In progress.
Abstraction	Pure virtual function virtual bool pay(double amount) = 0; inside abstract Payment class.	In progress.
Inheritance	UpiPayment, CardPayment, CashPayment inherit publicly from Payment.	In progress.
Runtime Polymorphism	Handled inside BookingService::processPayment(.., Payment* paymentMethod) via paymentMethod->pay().	In progress.

Compile-Time Polymorphism	Overloaded constructors in Movie class.	In progress.

Static Members	static int nextBookingId; in Booking class auto-increments for each new transaction.	In progress.
this Keyword	Used inside constructors across classes (this->seatNumber, this->title, this->name).	In progress.
Composition	Cinema → Screen, Screen → Seat, Show → ShowSeat.	In progress.
Aggregation	Show → Movie, Booking → ShowSeat.	In progress.
Association	Customer → BookingService.	In progress.
Edge Case 1	Duplicate booking attempt on seat G1 is rejected.	In progress.
Edge Case 2	Payment with "invalid@upi" triggers status revert and seat unlocking.	In progress.
Edge Case 3	cancelBooking() marks booking cancelled and makes seats AVAILABLE again.	In progress.
Edge Case 4	Requesting non-existent seat Z99 outputs a clear error without crashing.	In progress.

11. SOLID

PRINCIPLE	CONCEPT	USE IN CODE
Single Responsibility Principle (SRP)	A class should have one, and only one, reason to change.	BookingService should only focus on managing booking logic and pricing calculations. Printing layout, formatting receipts, or user interface presentation should be handled by a completely separate class (TicketPrinter). If ticket layout requirements change, you only edit TicketPrinter, leaving BookingService untouched.
Open-Closed Principle (OCP)	Software entities should be open for extension, but closed for modification.	I can add new functionality (such as a new payment method like NetBanking) by writing a new class, without altering or risking breaking existing classes like BookingService or CardPayment.
Liskov Substitution Principle (LSP)	Subtypes must be substitutable for their base types without breaking the application or requiring special-case checks.	Every payment subtype (CardPayment, CashPayment, NetBanking) must execute through a base pointer/reference (Payment* or PaymentC) effortlessly, without requiring extra initialization methods, flags, or type checks before invoking processPayment().
Interface Segregation Principle (ISP)	Clients should not be forced to depend upon interfaces they do not use.	Not all payment methods support online refunds (e.g., cash payments at a counter). Instead of forcing a dummy refund() implementation onto every payment class via the main Payment interface, split Refundable into a separate interface that only online payment methods implement.
Dependency Inversion Principle (DIP)	High-level modules should not depend on low-level modules. Both should depend on abstractions.	BookingService (high-level) should not directly instantiate low-level payment classes using new CardPayment(). Instead, BookingService accepts an abstract PaymentC dependency via method or constructor parameter injection.